



دانشگاه قم

دانشکده فنی و مهندسی

پروژه پایانی کارشناسی رشته مهندسی کامپیوتر

عنوان:

کاربرد احتمال در پیش بینی حملات سایبری و تحلیل رفتار ترافیک شبکه

استاد راهنما:

دکتر خیرالله رهسپار فرد

نگارنده:

امید شعبانعلی

مرداد ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

تقدیر:

از زحمات استاد محترم راهنما و همه اساتید عزیز تشکر و قدردانی می نمایم

چکیده:

با افزایش حملات هکری، تشخیص به موقع فعالیت‌های مشکوک در شبکه اهمیت زیادی یافته است. روش‌های سنتی که تنها حملات شناخته شده را شناسایی می‌کنند، در برابر روش‌های جدید کارایی لازم را ندارند. در این پروژه، یک سیستم تشخیص نفوذ طراحی شده است که به جای لیست کردن حملات، «رفتار غیرعادی» شبکه را تشخیص می‌دهد. برای این کار از مدل ریاضی **رگرسیون خطی** استفاده شده است.

در روند انجام این پروژه، ابتدا ترافیک معمولی شبکه ضبط گردید تا الگوی رفتار سیستم در حالت عادی استخراج شود؛ سپس با استفاده از **قاعده ۳-سیگما (3-Sigma Rule)** و محاسبه انحراف معیار، یک «آستانه مجاز» برای تعداد بسته‌های شبکه (PPS) به دست آمد. برای تست سیستم، با ابزار Zenmap به شبکه حمله شد. نتایج نشان داد که با شروع حمله، ترافیک از خط رگرسیون فاصله گرفته و سیستم سریعاً این ناهنجاری را تشخیص می‌دهد. در نهایت، سیستم به گونه‌ای تنظیم شد که به محض تشخیص حمله، یک پیام هشدار خودکار به اپلیکیشن **ایتا** ارسال کند تا کاربر در لحظه مطلع شود. این پروژه نشان می‌دهد که با بهره‌گیری از ریاضیات و آمار و مفاهیم احتمال می‌توان یک سیستم امنیتی کاربردی و هوشمند ساخت.

واژه‌های کلیدی: امنیت شبکه، تشخیص نفوذ، رگرسیون خطی، ناهنجاری ترافیک، n8n، پیام‌رسان ایتا.

فهرست مطالب

فصل ۱: مقدمه و نمای کلی سیستم	۱
۱-۱- مقدمه و اهمیت مانیتورینگ شبکه	۲
۱-۲- تشریح مسئله: چرا تشخیص نا هنجاری (Anomaly Detection)؟	۲
۱-۳- اهداف و سوالات اصلی پروژه	۳
۱-۳-۱- اهداف اصلی پروژه	۳
۱-۳-۲- سوالات کلیدی	۳
۱-۴- ضرورت و نوآوری پروژه	۳
۱-۵- معرفی ساختار گزارش	۴
فصل ۲: مفاهیم پایه و مبانی آماری پروژه	۵
۲-۱- واحد اندازه گیری ترافیک (PPS)	۶
۲-۲- رگرسیون خطی به زبان ساده	۶
۲-۳- تحلیل باقی مانده ها	۶
۲-۴- محاسبه آستانه بحرانی (The ۳-Sigma Rule)	۶
۲-۵- تحلیل سری های زمانی (Time-Series) و مفهوم خودهمبستگی در ترافیک شبکه	۷
۲-۶- تحلیل تطبیقی جامع انواع الگوریتم های رگرسیون در امنیت شبکه	۸
۲-۷- رویکردهای آمار کلاسیک و پیشینه تحقیق در سیستم های تشخیص نفوذ (Statistical IDS)	۹
۲-۷-۱- سیر تطور و پیشینه تحقیق (Literature Review)	۱۰
۲-۷-۲- مبانی نظری مدلسازی آماری ترافیک و پیش زمینه رگرسیون	۱۰
۲-۷-۳- تحلیل تفصیلی شاخص ها و آزمون های آماری در IDS	۱۱
۲-۷-۴- مقایسه تطبیقی جامع پژوهش های گذشته	۱۲
۲-۷-۵- چالش ها و محدودیت های سیستم های تشخیص نفوذ آماری	۱۳
۲-۸- ابزار پویش Nmap و تاثیر آن بر آمار	۱۳
فصل ۳: معرفی ابزارها و زیرساخت اجرایی سیستم	۱۵
۳-۱- معرفی نرم افزار Wireshark	۱۶
۳-۲- کتابخانه Scapy در پایتون	۱۷
۳-۳- ابزار پویش شبکه Zenmap (Nmap)	۱۷
۳-۴- پلتفرم اتوماسیون n8n	۱۸
۳-۵- پیام رسان ایتا و API ایتایار	۱۸
فصل ۴: پیاده سازی فنی و تشریح سیستم	۱۹

۴-۱	مقدمه فنی	۲۰
۴-۲	پیش‌پردازش و آماده‌سازی داده‌ها (Preprocessing)	۲۰
۴-۳	پایاده‌سازی مدل رگرسیون خطی	۲۰
۴-۴	محاسبه آستانه تشخیص ناهنجاری (Threshold)	۲۱
۴-۵	مکانیزم مانیتورینگ زنده و شنود پکت‌ها	۲۲
۴-۶	اتصال به شبکه و اتوماسیون هشدار (Integration)	۲۳
۴-۷	مکانیزم پاسخ فعال و مسدودسازی خودکار آی‌پی (Active Response & IP Blocking)	۲۴
۴-۷-۱	معماری و روند عملکرد واکنش خودکار	۲۴
۴-۸	توجیه معماری سیستم در انتخاب مدل پایه (Baseline) سبک جهت پردازش زنده	۲۵

فصل ۵: آزمایش، ارزیابی و نتیجه‌گیری

۵-۱	مقدمه	۲۸
۵-۲	معماری محیط آزمایشگاه و ابزارهای پایاده‌سازی	۲۸
۵-۳	سناریوی آزمایش	۲۹
۵-۴	مشاهده خروجی در حالت عادی	۲۹
۵-۵	اجرای حمله و واکنش سیستم	۲۹
۵-۶	ارزیابی آماری و تحلیل کارایی سیستم (Performance Evaluation)	۳۱
۵-۶-۱	ماتریس اغتشاش (Confusion Matrix)	۳۱
۵-۶-۲	معیارهای ارزیابی کارایی	۳۱
۵-۶-۳	پایاده‌سازی و ارزیابی مدل جایگزین (آزمون آماری کای-دو-۲۰۰۱ Ye & Chen)	۳۲
۵-۶-۴	مقایسه تطبیقی عملکرد مدل پیشنهادی و مدل کای-دو	۳۳
۵-۶-۵	تحلیل نتایج و نقد مقایسه‌ای	۳۳
۵-۷	دریافت هشدار در پیام‌رسان ایتا	۳۴
۵-۸	نتیجه‌گیری نهایی	۳۴
۵-۹	پیشنهادهای برای آینده	۳۴

منابع

۳۵	منابع
----	-------

پیوست‌ها

۳۸

فصل ۱:

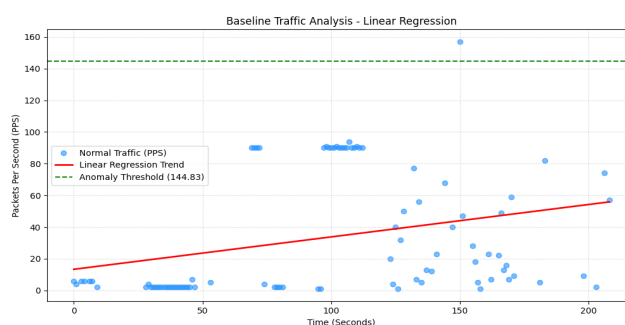
مقدمه و نمای کلی سیستم

۱-۱- مقدمه و اهمیت مانیتورینگ شبکه

امروزه شبکه‌های کامپیوتری ستون فقرات هر مجموعه‌ای هستند و پایداری آن‌ها مستقیماً بر عملکرد سیستم‌ها تاثیر می‌گذارد. با پیچیده‌تر شدن ابزارهای نفوذ، دیگر نمی‌توان تنها به فایروال‌های ساده تکیه کرد. مانیتورینگ یا پایش مداوم ترافیک، یکی از کلیدی‌ترین کارها برای حفظ امنیت است. اگر ما بدانیم در هر لحظه چه اتفاقی در شبکه می‌افتد، می‌توانیم قبل از اینکه یک نفوذ به فاجعه تبدیل شود، جلوی آن را بگیریم. در واقع امنیت شبکه و روش‌های مقابله با نفوذگران، نیازمند یک نگاه دقیق به آمارهای لحظه‌ای شبکه است [۱]

۱-۲- تشریح مسئله : چرا تشخیص نا هنجاری (Anomaly Detection)؟

اکثر سیستم‌های امنیتی قدیمی (مثل آنتی‌ویروس‌ها) بر اساس "امضا" کار می‌کنند؛ یعنی فقط حملاتی را می‌شناسند که قبلاً تعریف شده باشند. اما مشکل اینجاست که هکرها مدام از روش‌های جدید استفاده می‌کنند. در این پروژه، مسئله اصلی این است که ما به جای گشتن دنبال یک حمله خاص، به دنبال "رفتار غیرعادی" هستیم. ترافیک شبکه در حالت عادی یک نظم مشخص دارد، اما وقتی حمله‌ای مثل اسکن پورت‌ها^۲ اتفاق می‌افتد، این نظم به هم می‌خورد. هدف ما این است که با استفاده از مدل ریاضی رگرسیون خطی، این بی‌نظمی را شناسایی کنیم [۲]



عکس ۱-۱: تفاوت ترافیک منظم شبکه در حالت عادی و جهش ناگهانی پکت‌ها در زمان حمله

^۱ Anomaly Detection

^۲ Port Scanning

۱-۳- اهداف و سوالات اصلی پروژه

۱-۳-۱- اهداف اصلی پروژه

- ثبت رفتار عادی شبکه^۱: بررسی ترافیک واقعی شبکه (روی اینترفیس VMnet4^۲) برای فهمیدن اینکه در حالت معمولی چند پکت در ثانیه جابجا می‌شود.
- محاسبه آستانه مجاز (Threshold): پیدا کردن یک عدد مشخص به عنوان "مرز خطر" با استفاده از فرمول‌های ریاضی، به طوری که نوسانات کوچک شبکه باعث پیام اشتباه نشود.
- اتوماسیون اطلاع‌رسانی: ایجاد یک سیستم هوشمند که به محض دیدن ترافیک غیرعادی، خودش پیام هشدار را به اپلیکیشن ایتا بفرستد.

۱-۳-۲- سوالات کلیدی

۱. آیا یک فرمول ریاضی ساده (رگرسیون) می‌تواند جایگزین سیستم‌های سنگین و گران‌قیمت امنیتی برای تشخیص اسکن‌های شبکه شود؟
۲. چطور می‌توان ترافیک‌های زمینه‌ای ویندوز (مثل SSDP^۳) را از حملات واقعی تفکیک کرد؟
۳. سرعت پاسخگویی سیستم در زمان استفاده از ابزارهای اتوماسیون (n8n) چقدر است؟

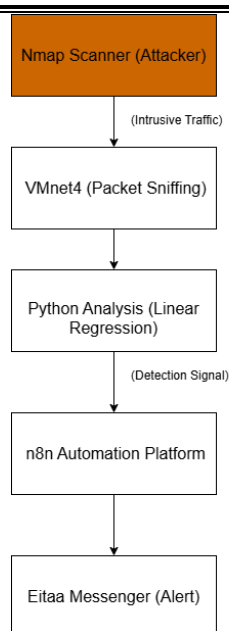
۴-۱- ضرورت و نوآوری پروژه

بسیاری از سیستم‌های هوش مصنوعی فعلی برای اجرا نیاز به سخت‌افزارهای قوی و سنگین دارند. نوآوری این پروژه در "سبک بودن" آن است [۳]. ما از مدلی استفاده کردیم که با کمترین فشار به CPU، بیشترین دقت را در تشخیص اسکن‌های پرسرعت دارد. همچنین، استفاده از پلتفرم n8n برای ارسال پیام به ایتا، این پروژه را به یک ابزار کاملاً کاربردی و مدرن تبدیل کرده است.

^۱ Baseline Traffic

^۲ VMware Virtual Network Adapter

^۳ Simple Service Discovery Protocol



عکس ۱-۲: نمودار کلی گردش کار سیستم (از ضبط پکت تا دریافت پیام در موبایل)

۵-۱- معرفی ساختار گزارش

این گزارش برای درک بهتر مراحل کار، به ۵ بخش تقسیم شده است:

- **فصل اول:** همین مقدمه و اهدافی که بررسی کردیم.
- **فصل دوم:** بررسی مبانی آماری و فرمول‌هایی که برای تحلیل ترافیک استفاده کردیم.
- **فصل سوم:** معرفی نرم‌افزارها و ابزارهایی مثل Scapy ، Nmap و n8n.
- **فصل چهارم:** جزئیات پیاده‌سازی، کدهای پایتون و نحوه تنظیم وب‌هوک.
- **فصل پنجم:** تست عملی سیستم، مشاهده نتایج در ایتا و نتیجه‌گیری نهایی.

فصل ۲:

مفاهیم پایه و مبانی آماری پروژه

۲-۱- واحد اندازه‌گیری ترافیک (PPS)

در این پروژه، به جای تمرکز بر حجم داده‌ها (بایت)، از واحد^۱ PPS استفاده کردیم. علت این انتخاب این است که در حملات پویش شبکه (Scanning)، مهاجم تعداد زیادی پکت کوچک برای بررسی پورت‌ها می‌فرستد. بنابراین، شمارش تعداد بسته‌ها در هر ثانیه، معیار بسیار دقیق‌تری برای تشخیص ناهنجاری نسبت به حجم ترافیک است [۴].

۲-۲- رگرسیون خطی به زبان ساده

رگرسیون خطی یکی از ساده‌ترین و در عین حال کاربردی‌ترین روش‌های آماری است. هدف این مدل، پیدا کردن رابطه‌ای بین زمان (محور افقی یا X) و تعداد بسته‌ها (محور عمودی یا Y) است. در واقع این مدل سعی می‌کند خطی را رسم کند که از نزدیک‌ترین فاصله ممکن نسبت به تمام نقاط ترافیک عادی عبور کند. معادله این خط به صورت $y = mx + b$ تعریف می‌شود که در آن m نشان‌دهنده شیب خط و b نشان‌دهنده عرض از مبدأ است.

۲-۳- تحلیل باقی‌مانده‌ها^۲

باقی‌مانده یا خطا، در واقع فاصله هر نقطه واقعی ترافیک تا خط رگرسیون است. در یک شبکه پایدار، این فاصله‌ها باید کم و ناچیز باشند. اما زمانی که حمله اتفاق می‌افتد، نقاط جدید ترافیک فاصله بسیار زیادی از خط پیدا می‌کنند. سیستم ما با محاسبه این فاصله‌ها متوجه می‌شود که رفتار شبکه از حالت پیش‌بینی شده خارج شده است [۵].

۲-۴- محاسبه آستانه بحرانی (The ۳-Sigma Rule)

یکی از بخش‌های مهم پروژه ما، تعیین عددی است که بالاتر از آن را به عنوان "حمله" در نظر

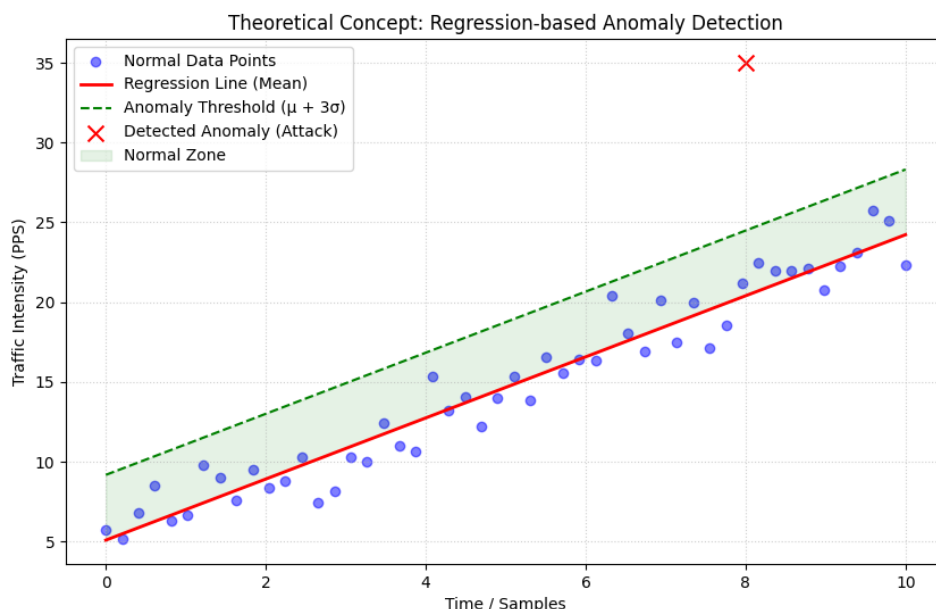
^۱ Packet Per Second

^۲ Residual Analysis

بگیریم. برای این کار از روش خطای استاندارد تخمین^۱ (SE) استفاده کردیم. طبق کدی که پیاده‌سازی شده، مراحل زیر طی شد: ابتدا میانگین ترافیک (Average PPS) به دست آمد. سپس میزان انحراف استاندارد خطا یا همان SE محاسبه شد. در نهایت، آستانه نهایی از فرمول زیر به دست آمد:

$$Threshold = Average + (3 * SE)$$

مبنای نظری انتخاب ضریب ۳ در این رابطه، **قانون ۳-سیگما** در توزیع نرمال است [۶]. طبق این اصل آماری، ۹۹.۷٪ از نوسانات ترافیک عادی شبکه درون محدوده $Average \pm 3SE$ قرار می‌گیرند [۷]. بنابراین، عبور ترافیک از این مرز با اطمینان آماری بالا (حدود ۹۹.۷٪) نشان‌دهنده وقوع ناهنجاری یا حمله در شبکه است



عکس ۲-۱: نمایش هندسی قانون ۳-سیگما و نحوه تعیین آستانه ناهنجاری در اطراف خط رگرسیون.

۲-۵- تحلیل سری‌های زمانی (Time-Series) و مفهوم خودهمبستگی در ترافیک شبکه

ترافیک شبکه بر حسب پارامترهایی نظیر تعداد بسته در ثانیه (PPS) یا حجم تبادل داده در طول

^۱ Standard Error of the Estimate

زمان، ماهیتاً یک سری زمانی (Time-Series) محسوب می‌شود. در تحلیل سری‌های زمانی، ترتیب زمانی مشاهده داده‌ها اهمیت بنیادی دارد، چرا که مقدار متغیر در لحظه t وابستگی شدیدی به مقادیر آن در لحظات قبلی دارد [۸].

در تحلیل‌های آماری کلاسیک، فرض بر این است که مشاهدات مستقل از یکدیگرند. با این حال، ترافیک شبکه رفتار خودهمبسته^۱ از خود نشان می‌دهد؛ یعنی یک صعود یا سقوط ناگهانی در ترافیک یک ثانیه، احتمالاً روی ثانیه‌های بعدی نیز تأثیرگذار است. مهم‌ترین چالش‌های تحلیل سری زمانی در ترافیک شبکه عبارت‌اند از:

- **غیرایستایی (Non-Stationarity):** میانگین و واریانس ترافیک شبکه در طول شبانه‌روز ثابت نیست و تابع ساعات کاری و بار سرور است.
 - **فصلی بودن (Seasonality):** الگوهای ترافیکی در فواصل زمانی مشخص (مثلاً ساعات پیک روزانه) به صورت تناوبی تکرار می‌شوند.
 - **نوسانات تصادفی (Noise):** نوسانات لحظه‌ای که ناشی از فعالیت‌های موقت کاربران است و نباید با حمله سایبری اشتباه گرفته شود.
- الگوریتم‌های پیشرفته سری زمانی نظیر $ARIMA^2$ یا هموارسازی نمایی با در نظر گرفتن این مولفه‌ها، توانایی پیش‌بینی دقیق‌تر رفتار آتی ترافیک را دارند [۹].

۶-۲- تحلیل تطبیقی جامع انواع الگوریتم‌های رگرسیون در امنیت شبکه

به منظور انتخاب مناسب‌ترین الگوریتم برای تحلیل ترافیک و تعیین آستانه ناهنجاری، عملکرد ریاضی و کاربردی انواع روش‌های رگرسیون در جدول ۲-۱ ارزیابی و مقایسه شده است [۵]:

^۱ Autocorrelated

^۲ Autoregressive Integrated Moving Average

جدول ۱-۲: مقایسه تطبیقی الگوریتم‌های رگرسیون، کاربردها و محدودیت‌های آن‌ها در تشخیص نفوذ

نوع رگرسیون	نحوه عملکرد ریاضی	کاربرد اصلی در امنیت شبکه	علت عدم انتخاب / محدودیت در پیاده‌سازی زنده
خطی ساده (Simple Linear)	برازش مستقیم‌ترین خط ($y = mx + b$) بر داده‌ها با کمترین مجموع مربعات خطا	تعیین خط روند کلی (Baseline Trend) و محاسبه حد آستانه ثابت [۵]	عدم پوشش وابستگی زمانی و رفتار فصلی
چندجمله‌ای (Polynomial)	برازش منحنی‌های درجه بالاتر ($y = a + bx + cx^2 + \dots$)	مدل‌سازی تغییرات غیرخطی ترافیک در ساعات مختلف روز	ریسک بالای پیش‌بینی‌های نادرست در فواصل دور (Overfitting) [۵]
لوجستیک (Logistic)	نگاشت داده‌ها به تابع سیگموئید جهت محاسبه احتمال وقوع	طبقه‌بندی باینری قطعی ترافیک (ترافیک عادی / ترافیک حمله)	نیازمندی به داده‌های برچسب‌دار در فاز آموزش [۱۰]
خطی منظم‌شده (Ridge / Lasso)	رگرسیون خطی به همراه جریمه ضرایب جهت جلوگیری از پیچیدگی	تحلیل ترافیک چندمتغیره با ویژگی‌های متعدد	عدم نیاز به ویژگی‌های متعدد در الگوی تک‌متغیره PPS [۵]
سری زمانی (ARIMA / SARIMA ^۱)	ترکیب خودرگرستی (AR^2) و میانگین متحرک (MA^3) روی داده‌های تفاضلی	پیش‌بینی دقیق ترافیک لحظه‌ای با لحاظ نمودن فصلی بودن	پیچیدگی محاسباتی بالا و تأخیر زمانی در تحلیل زنده [۹]

۷-۲- رویکردهای آمار کلاسیک و پیشینه تحقیق در سیستم‌های تشخیص نفوذ (Statistical IDS)

سیستم‌های تشخیص نفوذ مبتنی بر آمار^۴، از قدیمی‌ترین و در عین حال مستحکم‌ترین روش‌های حفظ امنیت در شبکه‌های کامپیوتر محسوب می‌شوند [۱۱]. ایده مرکزی این روش‌ها، مدل‌سازی رفتار

^۱ Seasonal Autoregressive Integrated Moving Average

^۲ AutoRegressive

^۳ Moving Average

^۴ Statistical Anomaly Detection Systems

"نرمال" شبکه با استفاده از توابع توزیع احتمال و متغیرهای تصادفی، و سپس شناسایی هرگونه انحراف معنی‌دار آماری به عنوان "رفتار ناهنجار" یا "حمله" است [۱۱].

۲-۷-۱- سیر تطور و پیشینه تحقیق (Literature Review)

نخستین پایه نظری سیستم‌های تشخیص نفوذ آماری در سال ۱۹۸۷ با ارائه مدل ایده‌آل IDS مطرح شد [۱۲]. در این مدل پیشنهاد شد که پروفایل رفتار کاربران و ترافیک شبکه را می‌توان با استفاده از شاخص‌های آماری نظیر میانگین، انحراف معیار و انحراف مطلق مدلسازی نمود [۱۲]. در سال‌های بعد، پژوهشگران جهت رفع چالش‌های نرخ هشدار کاذب (False Positive) و پیچیدگی پردازش زنده، الگوریتم‌های متعددی را توسعه دادند:

- روش‌های پارامتری کلاسیک: استفاده از آزمون‌های فرض آماری مانند Chi-Square و Z-Score جهت مقایسه توزیع لحظه‌ای ترافیک با پروفایل پایه^۱ [۱۳].
- مدل‌های سری زمانی و خودهمبستگی: بکارگیری مدل‌های ARIMA و Holt-Winters برای لحاظ نمودن وابستگی‌های زمانی و رفتارهای فصلی ترافیک [۱۴].
- روش‌های آماری سبک و چندمتغیره: استفاده از خطای استاندارد برآورد (S_e) و حدود آستانه مبتنی بر انحراف معیار جهت کاهش بار پردازشی CPU در سیستم‌های IPS زنده [۴].

۲-۷-۲- مبانی نظری مدلسازی آماری ترافیک و پیش‌زمینه رگرسیون

ترافیک شبکه از دیدگاه آمار، یک فرایند تصادفی^۲ است. اگر متغیر تصادفی X_t نشان‌دهنده تعداد بسته‌های دریافتی در ثانیه (PPS) در زمان t باشد، فرض پایه در اکثر مدل‌های آماری این است که X_t در شرایط عادی شبکه از یک توزیع احتمال مشخص (مانند توزیع نرمال یا پواسون) تبعیت می‌کند [۱۵].

^۱ Baseline

^۲ Stochastic Process

۱. مدل سازی با توزیع نرمال (Gaussian Traffic Model)

چنانچه حجم داده های نمونه کافی باشد، طبق "قضیه حد مرکزی ۱"، تابع چگالی احتمال (PDF^۲) ترافیک عادی شبکه به صورت زیر تعریف می شود [۱۶].

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

که در آن μ میانگین ترافیک و σ انحراف معیار ترافیک نرمال است [۱۶].

۲. فرآیند پواسون در ورود بسته ها (Poisson Process)

در شبکه هایی که نرخ ورود بسته ها مستقل از یکدیگر است، احتمال ورود k بسته در یک بازه زمانی t از رابطه زیر محاسبه می شود [۱۵]:

$$P(N(t) = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}$$

که در آن λ نرخ متوسط ورود بسته ها است.

۳-۷-۲- تحلیل تفصیلی شاخص ها و آزمون های آماری در IDS

۱. آزمون نمره استاندارد (Z-Score Analysis)

آزمون $Z - Score$ میزان فاصله یک داده لحظه ای x از میانگین μ را بر حسب واحد انحراف معیار σ می سنجد [۱۶]:

$$Z = \frac{x - \mu}{\sigma}$$

- معیار تصمیم گیری: اگر $|Z| > 3$ باشد، با احتمال ۹۹.۷٪ داده مورد نظر یک ناهنجاری آماری تلقی می گردد [۱۶]

۲. آزمون مجذور کای (χ^2 Test)

^۱ Central Limit Theorem

^۲ Probability Density Function

این آزمون جهت مقایسه توزیع فراوانی بسته‌های مشاهده شده زنده (O_i) با توزیع فراوانی مورد انتظار بر اساس خط پایه (E_i) بکار می‌رود [۱۳].

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

هرچه مقدار χ^2 از مقدار بحرانی جدول توزیع بیشتر باشد، احتمال وقوع تغییر در الگوی ترافیک و حمله بیشتر است [۱۳].

۳. خطای استاندارد برآورد (S_e) و آستانه ۳-سیگما در رگرسیون

در مدل‌های رگرسیونی، به جای استفاده از انحراف معیار ساده حول میانگین، انحراف داده‌ها حول خط روند رگرسیون سنجیده می‌شود [۹]. خطای استاندارد برآورد (S_e) به صورت زیر تعریف می‌گردد [۵]:

$$S_e = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - 2}}$$

که در آن y_i مقدار واقعی PPS، $\hat{y}_i$ مقدار پیش‌بینی شده توسط خط رگرسیون و n تعداد نمونه‌ها است [۵]. حد آستانه بحرانی تشخیص نفوذ بر اساس قاعده ۳-سیگما به صورت زیر است [۵]:

$$Threshold = \bar{Y} + 3 \times S_e$$

این رابطه ضمانت می‌کند که نرخ آلامر کاذب در شرایط عادی کمتر از ۰.۳٪ باشد [۵].

۴-۷-۲- مقایسه تطبیقی جامع پژوهش‌های گذشته

جدول ۲-۲ خلاصه‌ای از رویکردها، مفاهیم آماری و نتایج پژوهشگران مختلف در حوزه سیستم‌های تشخیص نفوذ آماری را ارائه می‌دهد:

جدول ۲-۲: مقایسه تطبیقی پژوهش‌های پیشین در حوزه سیستم‌های تشخیص نفوذ آماری و موقعیت طرح پیشنهادی

مرجع / پژوهش	الگوریتم / روش آماری	پارامتر ارزیابی	مزایای اصلی	محدودیت‌ها و چالش‌ها
Denning [۱۲]	پرو فایل آماری + انحراف معیار	نرخ ورود / حجم داده	پایه‌گذار مفاهیم اولیه IDS آماری	عدم پشتیبانی از رفتارهای پویا و فصلی
Ye et al. [۱۳]	آزمون χ^2 و Z-score	فرکانس حوادث شبکه	دقت بالا در شناسایی انحرافات کلی	حساسیت بالا به نویز و ایجاد آلارم کاذب
Brockwell & Davis [۱۴]	مدل سری زمانی ARIMA	نرخ پکت در زمان (PPS)	مدلسازی دقیق رفتارهای فصلی و خودهمبسته	پیچیدگی محاسباتی بالا و نامناسب برای real-time
Scarfone & Mell [۴]	آستانه‌گذاری آماری S_e	نرخ پکت بر ثانیه (PPS)	پردازش سبک، سرعت بسیار بالا در لایه IPS	نیازمند بازتنظیم آستانه در تغییر شبکه
طرح پیشنهادی (پروژه حاضر)	رگرسیون خطی + قاعده $Response + 3S_e$	PPS لحظه‌ای + IP Tracking	پاسخ فعال خودکار، پردازش زنده، اتصال به n8n	نیازمند فاز آموزش ابتدایی برای ثبت baseline

۲-۷-۵- چالش‌ها و محدودیت‌های سیستم‌های تشخیص نفوذ آماری

با وجود سرعت و سادگی سیستم‌های آماری، این سیستم‌ها با چالش‌های زیر مواجه هستند:

- پدیده Concept Drift^۱: با گذشت زمان و تغییر رفتار کاربران، خط پایه^۲ قدیمی شده و باید بازسازی شود [۴].
- ترافیک‌های ناگهانی مجاز (Burst Traffic): رویدادهایی مانند دانلود یک فایل حجم بالا ممکن است به اشتباه حمله تشخیص داده شود که استفاده از ضریب ۳-سیگما این خطر را کاهش می‌دهد [۵].

۲-۸- ابزار پویش Nmap و تاثیر آن بر آمار

ابزار Nmap در حالت‌های تهاجمی (مثل سویچ T4-) بسته‌ها را با سرعت بسیار بالا و بدون وقفه در شبکه می‌فرستد [۲]. این کار باعث می‌شود مقدار ترافیک در مدل رگرسیون ما به ناگهان جهش کند و

^۱ تغییر تدریجی الگوی شبکه

^۲ Baseline

از آستانه یا همان Threshold که در بخش قبل حساب کردیم، عبور کند [۵]. سیستم در این لحظه با مقایسه ترافیک لحظه‌ای و آستانه تعیین شده، وضعیت حمله را شناسایی کرده و هشدار صادر می‌کند.

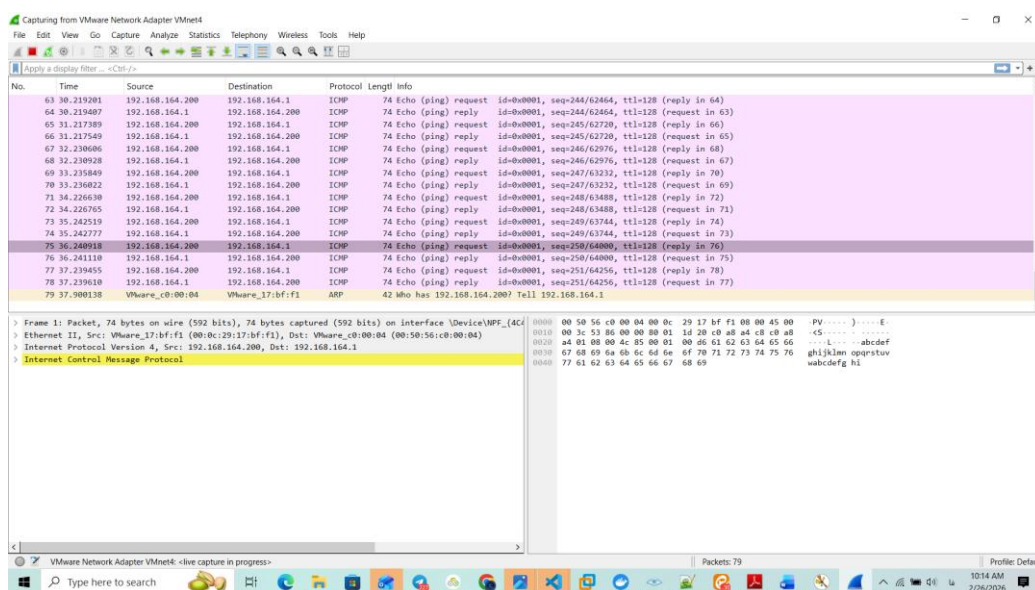
فصل ۳:

معرفی ابزارها و زیرساخت اجرایی

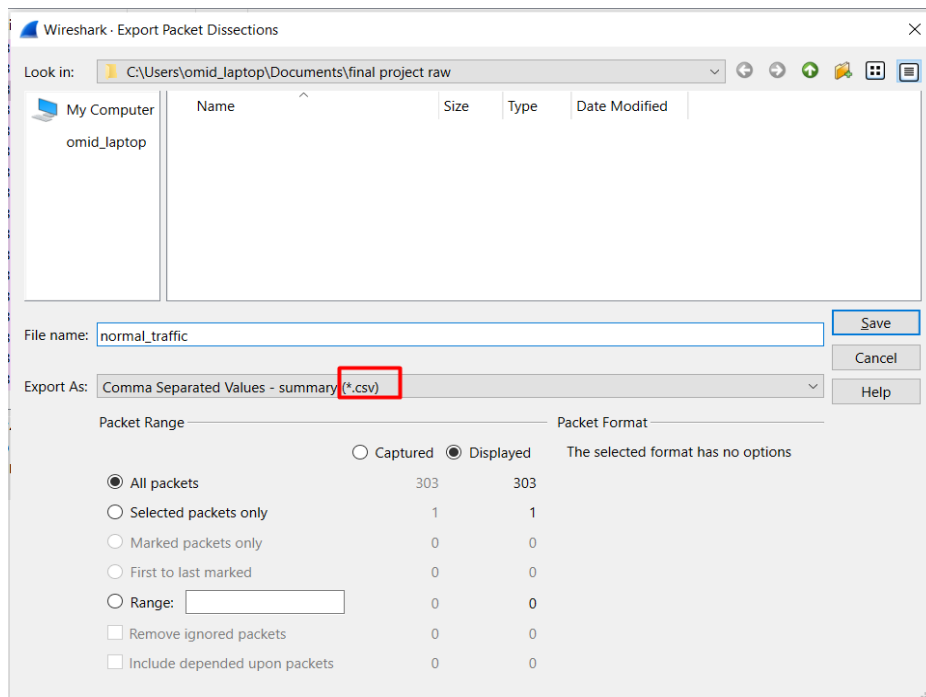
سیستم

۳-۱- معرفی نرم افزار Wireshark

ما در این پروژه برای جمع‌آوری داده‌های اولیه (Baseline) از وایرشارک استفاده کردیم [۱۷]. این ابزار به ما اجازه داد تا بفهمیم در شبکه مجازی ما (VMnet4) چه پکت‌هایی در حال رد و بدل شدن است. خروجی این ابزار به صورت فایل CSV استخراج شد تا برای پایتون قابل فهم باشد.



عکس ۳-۱: محیط نرم افزار Wireshark در حال پایش و ثبت ترافیک اولیه (Baseline) شبکه در اینترنت فیس ۴ VMnet



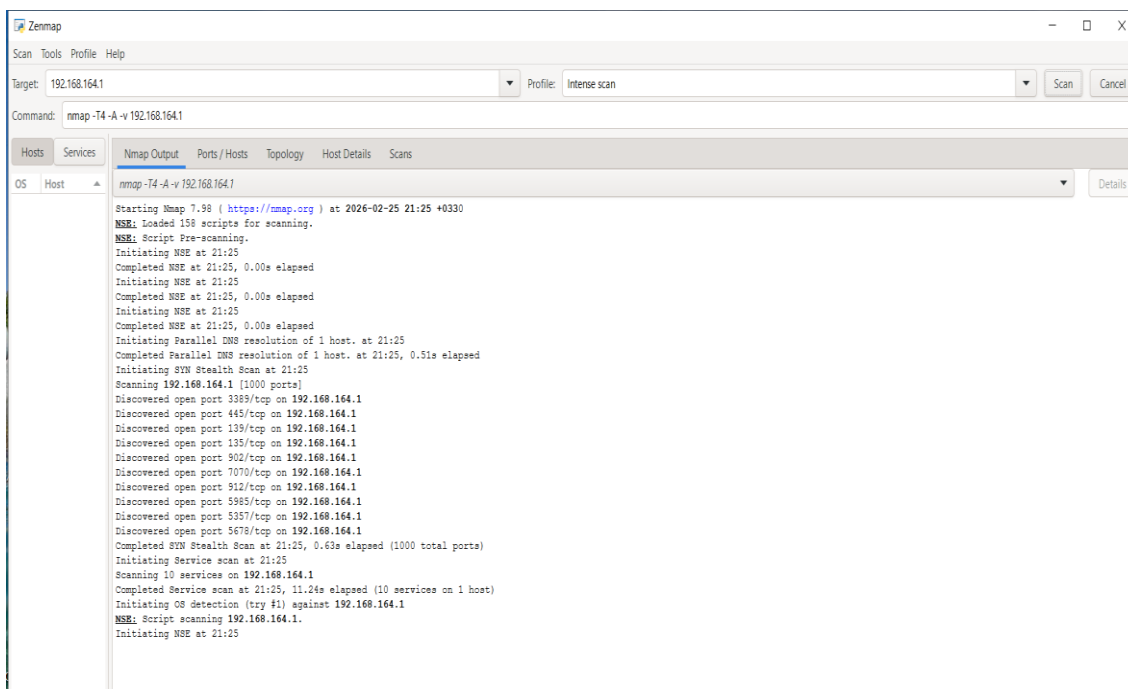
عکس ۳-۲: فرآیند شنود داده‌های شبکه و آماده‌سازی فایل خروجی CSV برای پردازش در مدل رگرسیون

۳-۲- کتابخانه Scapy^۱ در پایتون

قلب بخش مانیتورینگ زنده ما، کتابخانه Scapy است. برخلاف کتابخانه‌های معمولی، Scapy به ما اجازه می‌دهد مستقیماً به کارت شبکه وصل شویم و پکت‌ها را در لحظه "شنود (Sniff)" کنیم [۱۸]. ما از این ابزار برای شمارش پکت‌ها در بازه‌های ۱ ثانیه‌ای استفاده کردیم.

۳-۳- ابزار پویش شبکه (Nmap) Zenmap

برای اینکه سیستم را تست کنیم، نیاز به یک "مهاجم" داشتیم Zenmap. نسخه گرافیکی Nmap است که با تنظیم آن روی حالت **Intense Scan**، ترافیک سنگینی ایجاد کردیم تا ببینیم آیا مدل رگرسیون ما متوجه این تغییر ناگهانی می‌شود یا خیر.

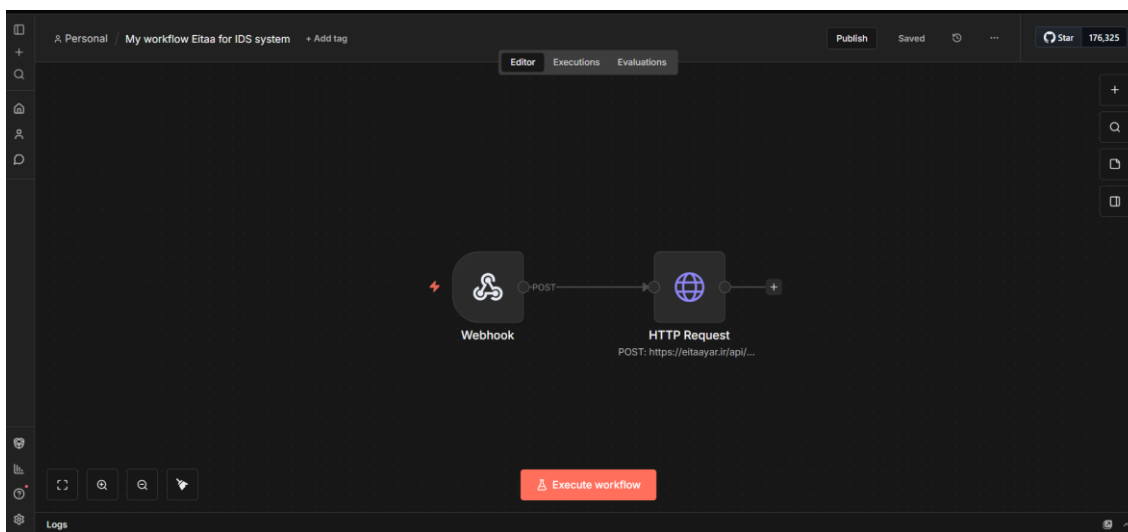


عکس ۳-۳: محیط نرم‌افزار Zenmap در حال اجرای اسکن روی آی‌پی هدف

^۱ Scapy Packet Inspection

۴-۳- پلتفرم اتوماسیون n8n^۱

یکی از بخش‌های مدرن این پروژه استفاده از n8n است [۱۹]. این ابزار به عنوان یک "واسط" عمل می‌کند. وقتی پایتون حمله را تشخیص می‌دهد، یک سیگنال به n8n می‌فرستد و n8n وظیفه دارد این سیگنال را به پیام قابل فهم برای "بات ایتا" تبدیل کند. این کار باعث می‌شود سیستم ما بدون نیاز به مانیتور کردن مداوم کنسول پایتون، هشدارها را به موبایل ما بفرستد.



عکس ۳-۴: نمای محیط کاربری n8n و گره‌های (Nodes) متصل شده به هم

۵-۳- پیام‌رسان ایتا و API ایتایار

برای بخش اطلاع‌رسانی، از پیام‌رسان بومی ایتا استفاده شد. با استفاده از قابلیت Webhook^۲ و اتصال آن به n8n، توانستیم یک کانال ارتباطی امن برای دریافت هشدارهای لحظه‌ای ایجاد کنیم.

^۱ n8n Automation Engine

^۲ Webhook HTTP Endpoint

فصل ۴:

پیاده‌سازی فنی و تشریح سیستم

۱-۴- مقدمه فنی

در این فصل به بررسی جزئیات پیاده‌سازی کدها، نحوه پردازش داده‌های خام و اتصال بخش‌های مختلف سیستم به یکدیگر می‌پردازیم. پیاده‌سازی این پروژه شامل سه بخش اصلی است: استخراج ویژگی‌ها از ترافیک، مدل‌سازی ریاضی و در نهایت مانیتورینگ زنده.

۲-۴- پیش‌پردازش و آماده‌سازی داده‌ها (Preprocessing)

داده‌های خام که از نرم‌افزار وایرشارک به صورت فایل CSV استخراج شده بودند، توسط کتابخانه pandas در پایتون فراخوانی شدند. از آنجایی که ترافیک شبکه در لحظات مختلف نوسان دارد، ما داده‌ها را بر اساس ستون زمان (Time) گروه‌بندی کردیم تا تعداد بسته‌ها در هر ثانیه یا همان PPS مشخص شود. در این مرحله، نوسانات ناگهانی ترافیک^۱ نیز به عنوان بخشی از رفتار طبیعی شبکه در داده‌های آموزشی حفظ شدند تا مدل نسبت به فعالیت‌های عادی پر حجم حساسیت کاذب نشان نداده و آستانه تشخیص (Threshold) با دقت بیشتری تعیین گردد [۲۰]. این کار به ما کمک کرد تا یک دید کلی از وضعیت ترافیک نرمال شبکه به دست آوریم.

```
# Preprocessing: Round time to nearest second
# This allows us to count packets per second (PPS)
df['Time'] = df['Time'].astype(float).round(0)

# Group by 'Time' to get the count of packets in each second
pps_data = df.groupby('Time').size().reset_index(name='Packet_Count')
```

شرح برنامه ۴-۱: در این قطعه کد، از کتابخانه pandas برای گروه‌بندی داده‌های خام و استخراج نرخ پکت در ثانیه استفاده شده است.

۳-۴- پیاده‌سازی مدل رگرسیون خطی

برای مدل‌سازی ترافیک، از کلاس LinearRegression در کتابخانه scikit-learn استفاده شد. مدل سعی می‌کند رابطه‌ای بین زمان (محور x) و تعداد بسته‌ها (محور y) پیدا کند. این رابطه در ساده‌ترین

^۱ Burst Traffic

حالت به صورت فرمول $y = wx + b$ تعریف می‌شود.

پس از آموزش دیدن مدل روی داده‌های عادی، "باقی‌مانده‌ها" یا همان Residuals محاسبه شدند. باقی‌مانده در واقع تفاضل بین مقدار واقعی مشاهده شده در شبکه و مقداری است که مدل ما پیش‌بینی کرده است. فرمول آن به زبان ساده به این صورت است:

مقدار پیش‌بینی شده - مقدار واقعی = باقی‌مانده

مدل رگرسیون مورد استفاده، با کمترین میزان خطای میانگین مربعات (MSE) بر روی داده‌های آموزشی برازش شد تا پیش‌بینی ترافیک نرمال با دقت حداکثری انجام پذیرد و انحرافات جزئی نیز قابل شناسایی باشند.

```
# Initialize and fit the Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Predict values to calculate residuals
y_pred = model.predict(X)
```

شرح برنامه ۴-۲: برازش مدل رگرسیون خطی بر روی داده‌های آموزشی جهت استخراج خط روند ترافیک نرمال.

۴-۴- محاسبه آستانه تشخیص ناهنجاری (Threshold)

مهم‌ترین بخش آماری این پروژه، محاسبه "خطای استاندارد برآورد" یا همان Se است. این عدد به ما می‌گوید که داده‌های عادی شبکه چقدر از خط اصلی رگرسیون فاصله دارند. برای تعیین آستانه نهایی و مرز تشخیص حمله، از میانگین ترافیک به علاوه ۳ برابر خطای استاندارد استفاده کردیم. این کار باعث می‌شود دقت سیستم بالا برود و نوسانات معمولی شبکه به اشتباه "حمله" در نظر گرفته نشوند (کاهش هشدارهای اشتباه). فرمول نهایی ما به این صورت بود:

$$\text{Threshold} = \text{Average} + (3 * \text{Se})$$

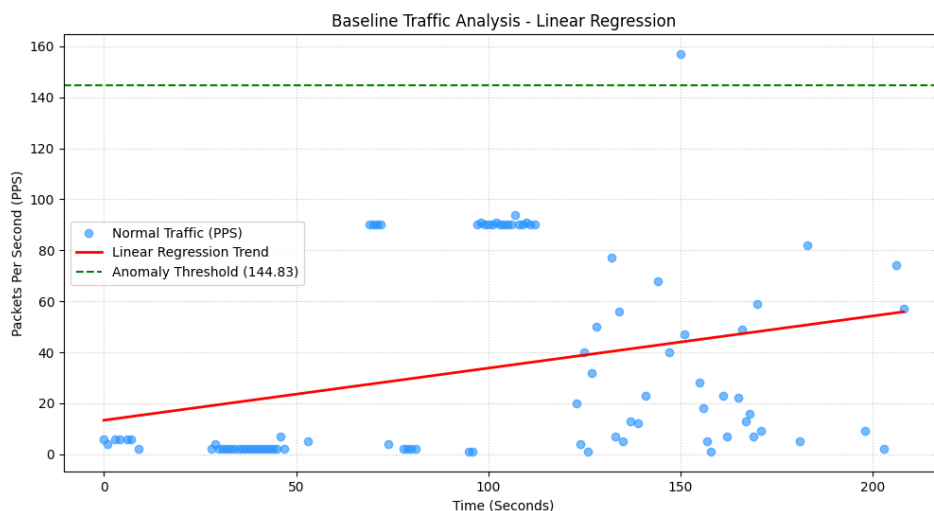
```
# Calculate Standard Error of the Estimate (Se)
# Se = sqrt( sum( (y_actual - y_pred)^2 ) / (n - 2) )
residuals = y - y_pred
se = np.sqrt(np.sum(residuals**2) / (len(y) - 2))

# Calculate Threshold (3-sigma rule: 3 * Se above the regression line)
# We use this as our anomaly detection boundary
avg_pps = np.mean(y)
threshold = avg_pps + (3 * se)
```

شرح برنامه ۴-۳: محاسبه خطای استاندارد برآورد و تعیین آستانه نهایی تشخیص حمله با استفاده از قانون ۳-سیگما.

طبق محاسبات انجام شده روی داده‌های واقعی این پروژه، عدد ۱۴۴.۸۳ به عنوان مرز بحرانی یا

آستانه نهایی به دست آمد.



شکل ۴-۱: نمودار تحلیل رگرسیون ترافیک واقعی و تعیین آستانه بحرانی (Threshold) بر اساس داده‌های شبکه.

توضیحات شکل: در این نمودار، نقاط آبی نشان‌دهنده تعداد بسته‌ها در ثانیه (PPS) در طول زمان هستند. خط قرمز، روند کلی ترافیک نرمال را مدل‌سازی کرده و خط چین سبز، آستانه بحرانی (۱۴۴.۸۳) را نشان می‌دهد که با فرمول $3S_e + Average$ به دست آمده است. عبور نقطه ناهنجار از مرز مشخص شده، لحظه دقیق شناسایی حمله Nmap را تایید می‌کند.

۴-۵- مکانیزم مانیتورینگ زنده و شنود پکت‌ها

در این بخش از کتابخانه قدرتمند Scapy استفاده شده است. اسکریپت اصلی پروژه با استفاده از تابع sniff روی کارت شبکه مجازی، بسته‌ها را در بازه‌های یک ثانیه‌ای می‌شمارد. منطق تشخیص حمله در کد بسیار ساده و سریع طراحی شده است؛ به این صورت که اگر تعداد بسته‌های شمارش شده در یک ثانیه (PPS لحظه‌ای) از عدد آستانه (۱۴۴.۸۳) بیشتر شود، سیستم وضعیت را "بحرانی" تشخیص داده و بلافاصله تابع ارسال هشدار را اجرا می‌کند.

```
while True:
    packet_count = 0
    ip_counter.clear()

    # Sniff packets for 1 second
    scapy.sniff(iface=INTERFACE, prn=process_packet, timeout=1, store=0)

    current_pps = packet_count
    print(f"Current Traffic: {current_pps} PPS", end='\n')

    if current_pps > THRESHOLD:
        # Capture accurate system time upon attack detection
        attack_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        print(f"\n[!] ANOMALY DETECTED at [{attack_time}]! Traffic: {current_pps} PPS")

        attacker_ip = "Unknown"
        if ip_counter:
            attacker_ip = ip_counter.most_common(1)[0][0]
            print(f"[!] Attacker IP identified: {attacker_ip}")

        # Active response execution
        block_ip(attacker_ip)

        send_alert(current_pps, attacker_ip)
        time.sleep(5)
```

شرح برنامه ۴-۴: مانیتورینگ بلادرنگ اینترنتی شبکه و مقایسه PPS لحظه‌ای با آستانه بحرانی جهت تشخیص ناهنجاری.

```
Administrator: C:\Windows\system32\cmd.exe - python detect_attack.py
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>cd C:\Drive D Backup 1405-03-5\Omid\0000 0000000\000 7\Final Project\project\src

C:\Drive D Backup 1405-03-5\Omid\0000 0000000\000 7\Final Project\project\src>python detect_attack.py
[*] Monitoring started on VMware Network Adapter VMnet4...
[*] Threshold is set to: 144.83 PPS
Current Traffic: 184 PPS
[!] ANOMALY DETECTED at [2026-07-28 20:11:06]! Traffic: 184 PPS
[!] Attacker IP identified: 192.168.164.200
[+] Successfully blocked ALL traffic for IP 192.168.164.200 in Windows Firewall.
[+] Alert sent to n8n! Traffic: 184 PPS | Target IP: 192.168.164.200
```

عکس ۴-۲: اسکرین‌شات از محیط خط فرمان (cmd) در لحظه شناسایی ناهنجاری و صدور هشدار سیستمی.

۶-۴- اتصال به شبکه و اتوماسیون هشدار (Integration)

زمانی که حمله تشخیص داده می‌شود، سیستم یک درخواست به سمت سرور n8n ارسال می‌کند. این داده‌ها شامل اطلاعاتی مثل زمان دقیق حمله و تعداد پکت‌ها است. پلتفرم n8n با دریافت این اطلاعات، بلافاصله از طریق یک رابط (Webhook) پیام را به بات طراحی شده در پیام‌رسان ایتا منتقل می‌کند. این زنجیره باعث می‌شود که مدیر شبکه بدون نیاز به چک کردن مداوم سیستم، از طریق موبایل خود در جریان حملات قرار بگیرد.

```
def send_alert(pps, attacker_ip):
    payload = {
        "method": "Linear Regression Model",
        "traffic": pps,
        "threshold": THRESHOLD,
        "status": "Attack Detected & Blocked",
        "message": "Intrusion detected by Linear Regression Model",
        "student": "Omid",
        "network": "VMware VMnet4",
        "blocked_ip": attacker_ip,
        "action": "Active Response - Firewall Block"
    }
```

شرح برنامه ۴-۵: متد ارسال داده‌های ناهنجاری به وب‌هوک n8n جهت اطلاع‌رسانی خودکار در پیام‌رسان ایتا

۷-۴- مکانیزم پاسخ فعال^۱ و مسدودسازی خودکار آی‌پی (Active Response & IP Blocking)

در سامانه‌های تشخیص نفوذ سنتی (Passive IDS)، سیستم تنها پس از شناسایی آنومالی اقدام به ثبت گزارش و ارسال هشدار به مدیر شبکه می‌نماید. اما در سناریوهای حملات منع خدمت (DoS/DDoS) یا اکسپلویت‌های سریع شبکه، فاصله زمانی میان اعلان هشدار تا اقدام دستی مدیر شبکه می‌تواند منجر به آسیب‌پذیری شدید یا از دسترس خارج شدن سرویس‌ها گردد. جهت رفع این چالش، سامانه طراحی شده در این پروژه به مازول پاسخ فعال (Active Response) مجهز گردید تا عملیات پیشگیری از نفوذ (IPS^۲) به صورت خودکار انجام پذیرد. [۱۱]

۱-۷-۴- معماری و روند عملکرد واکنش خودکار

فرآیند شناسایی تا مسدودسازی خودکار آی‌پی مهاجم طی ۶ گام عملیاتی به شرح زیر انجام می‌پذیرد:

۱. تحلیل عمیق بسته‌ها (Packet Inspection): تابع process_packet حین شنود ترافیک

کارت شبکه (VMware VMnet4)، تمامی بسته‌های لایه شبکه را بررسی کرده و آدرس آی‌پی مبدأ (src_ip) را استخراج می‌نماید.

^۱ Active Response

^۲ Intrusion Prevention System

۲. **شمارش بر اساس آدرس مبدأ:** با به‌کارگیری ساختار داده Counter از کتابخانه collections

در پایتون، تعداد بسته‌های ارسالی توسط هر آدرس IP به صورت مجزا در بازه‌های زمانی یک‌ثانیه‌ای ثبت می‌شود.

۳. **ارزیابی حد آستانه (Threshold Evaluation):** چنانچه مجموع بسته‌های دریافتی در یک

ثانیه از حد آستانه مدل رگرسیون خطی (معادل ۱۴۴.۸۳ PPS) فراتر رود، وضعیت ناهنجاری صادر می‌شود.

۴. **شناسایی منبع اصلی حمله:** الگوریتم با فراخوانی متد most_common(1) آدرس آی‌پی

که بیشترین سهم را در تولید ترافیک غیرمجاز در آن ثانیه داشته به عنوان آی‌پی مهاجم تفکیک می‌کند.

۵. **تزریق قانون به فایروال (Firewall Rule Injection):** ماژول block_ip بر اساس نوع

سیستم عامل میزبان، دستور بومی فایروال را صادر می‌کند [۲۱]:

در محیط ویندوز:

اجرای دستور زیر در سطح دسترسی Administrator جهت مسدودسازی ترافیک ورودی:

```
netsh advfirewall firewall add rule name="IDS_AUTO_BLOCK_[IP]" dir=in
action=block remoteip=[IP]
```

در محیط لینوکس:

اجرای دستور زیر در جدول iptables :

```
iptables -A INPUT -s [IP] -j DROP
```

۶. **مدیریت وضعیت (State Management) و اعلام هشدار:** جهت جلوگیری از کندی

فایروال ناشی از ثبت قوانین تکراری، آدرس‌های مسدودشده در مجموعه‌ای موقت

(blocked_ips) ذخیره می‌گردند. در نهایت، داده‌های تحلیل به همراه آدرس IP مسدودشده

در قالب یک محموله JSON به وب‌هوک n8n ارسال می‌شود تا گزارش آن به پیام رسان منتقل گردد.

۸-۴. **توجیه معماری سیستم در انتخاب مدل پایه (Baseline) سبک جهت**

پردازش زنده

اگرچه الگوریتم‌های پیچیده‌تر سری زمانی مانند ARIMA یا مدل‌های یادگیری عمیق توانایی

بالاتری در کشف الگوهای پیچیده و غیرخطی ترافیک دارند، اما در سیستم‌های پیشگیری از نفوذ (IPS)

زنده، زمان پاسخ‌دهی (Latency) و مصرف منابع پردازشی (CPU/RAM Overhead) دو فاکتور بسیار بحرانی هستند.

در این پروژه، استفاده از رگرسیون خطی ساده به همراه آستانه‌گذاری 3σ به دلایل زیر به عنوان مدل پایه انتخاب شده است [۴]:

- **سرعت محاسباتی در حد میکروثانیه:** محاسبه حد آستانه در رگرسیون خطی پیچیدگی زمانی $O(N)$ دارد که امکان تحلیل زنده هزاران پکت در ثانیه را بدون ایجاد گلوگاه (Bottleneck) فراهم می‌سازد [۲۲].
- **قابلیت پیاده‌سازی ساده در لایه پاسخ فعال (Active Response):** پایین بودن بار پردازشی مدل، به اسکریپت `detect_attack.py` اجازه می‌دهد تا بدون افت فریم یا از دست رفتن پکت‌ها (Packet Loss)، بلافاصله پس از شناسایی حمله، دستورات مسدودسازی را به فایروال سیستم‌عامل (`netsh` در ویندوز یا `iptables` در لینوکس) تزریق کند [۱۷].
- **ارائه الگوی Baseline سریع:** این مدل یک نقطه مرجع (Baseline) شفاف و قابل تفسیر ایجاد می‌کند تا در کاربری‌های صنعتی بتوان کارایی سیستم را پیش از بکارگیری مدل‌های سنگین‌تر ارزیابی نمود [۴].

^۱ Network Shell

^۲ IP Tables Utility

فصل ۵:

آزمایش، ارزیابی و نتیجه‌گیری

۱-۵- مقدمه

در این فصل، کارایی و صحت عملکرد سیستم تشخیص و پاسخ خودکار به نفوذ (IDS/IPS) توسعه‌یافته بر پایه الگوریتم رگرسیون خطی و قاعده آستانه‌گذاری ۳-سیگما به همراه ارزیابی عملیاتی آن در محیط آزمایشگاهی بررسی می‌شود. پیاده‌سازی نرم‌افزاری سیستم شامل دو ماژول اصلی پایتون یعنی train_model.py (جهت استخراج خط پایه و محاسبه آستانه بحرانی) و ids_agent.py (جهت شنود زنده ترافیک، مسدودسازی دوطرفه فایروال و ارسال هشدار به وب‌هوک n8n) می‌باشد.

۲-۵- معماری محیط آزمایشگاه و ابزارهای پیاده‌سازی

آزمایش در یک محیط مجزاسازی‌شده مجازی با مشخصات فنی درج‌شده در جدول ۵-۱ انجام پذیرفت:

جدول ۵-۱: مشخصات فنی و ساختار ابزارهای آزمایشگاهی

لایه / بخش	ابزار / مشخصات فنی	نقش و وظیفه عملیاتی
کارت شبکه مانیتور شده	VMware Network Adapter VMnet۴	رابط شبکه‌ای مجازی تحت نظارت زنده اسکریپت پایتون
ماژول آموزش (Training)	train_model.py (scikit-learn, pandas)	پردازش داده‌های normal_traffic.csv و محاسبه آستانه
ماژول آشکارسازی و پاسخ	ids_agent.py (Scapy, Requests)	شنود زنده ۱ ثانیه‌ای، استخراج IP مهاجم و انسداد فایروال
سیستم مهاجم (Attacker)	Zenmap (Nmap GUI)	اجرای پویش تهاجمی شبکه (Intense Scan)
بستر اتوماسیون و هشدار	n8n Webhook Engine & Eitaa Bot	دریافت JSON Payload و انتقال آنی پیام هشدار

۳-۵- سناریوی آزمایش

- برای تست سیستم، یک محیط آزمایشگاهی مجازی شامل دو بخش زیر ایجاد شد :
۱. **سیستم هدف (Target):** یک ماشین مجازی (در این جا ویندوز ۷) که کارت شبکه آن (VMnet4) توسط اسکریپت پایتون ما مانیتور می‌شود.
 ۲. **سیستم مهاجم (Attacker):** استفاده از ابزار Zenmap برای ارسال پکت‌های پویش شبکه (Intense Scan) به سمت سیستم هدف.

۴-۵- مشاهده خروجی در حالت عادی

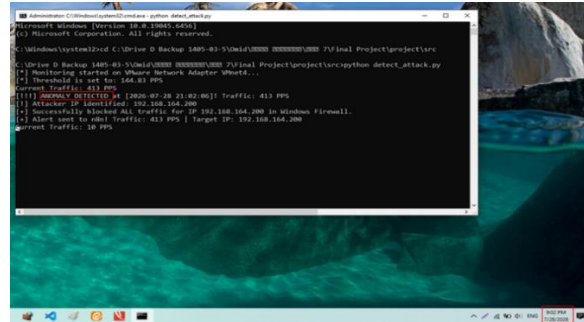
در ابتدای کار، سیستم در شرایطی قرار گرفت که فقط فعالیت‌های معمولی شبکه (مثل بسته‌های شناسایی ویندوز) در جریان بود. در این حالت، تعداد بسته‌ها در هر ثانیه (PPS) بسیار پایین‌تر از آستانه ۱۴۴.۸۳ بود. نمودار خروجی پایتون نشان داد که تمام نقاط ترافیکی در اطراف خط رگرسیون قرار دارند و هیچ هشدار اشتباهی صادر نشد.

۵-۵- اجرای حمله و واکنش سیستم

به محض شروع اسکن توسط Zenmap، تعداد بسته‌های ارسالی در ثانیه به شدت افزایش یافت و به عددی بالای ۴۰۰ بسته در ثانیه رسید. توالی زمانی^۱ پاسخ‌دهی سیستم در این لحظه به‌صورت زیر اجرا گردید:

۱. **کشف ناهنجاری (Detection Phase - ثانیه T۰):** سیستم پایتون در پنجره زمانی ۱ ثانیه‌ای، نرخ ترافیک ۴۱۳ PPS را ثبت کرد. از آنجا که این مقدار از آستانه ۱۴۴.۸۳ PPS عبور کرد، بلافاصله پیام "ANOMALY DETECTED" به همراه مشخصات ترافیک در کنسول چاپ شد.

^۱ Chronology

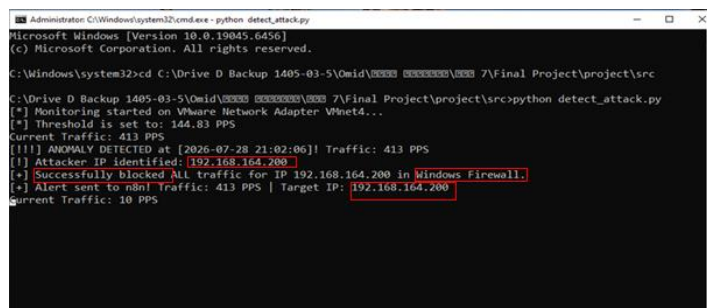


شکل ۵-۱: تشخیص ناهنجاری در محیط خط فرمان پایتون بلافاصله پس از شروع حمله.

۲. ارسال هشدار و فعال سازی پاسخ Alerting & Mitigation Trigger - T₀ ثانیه +

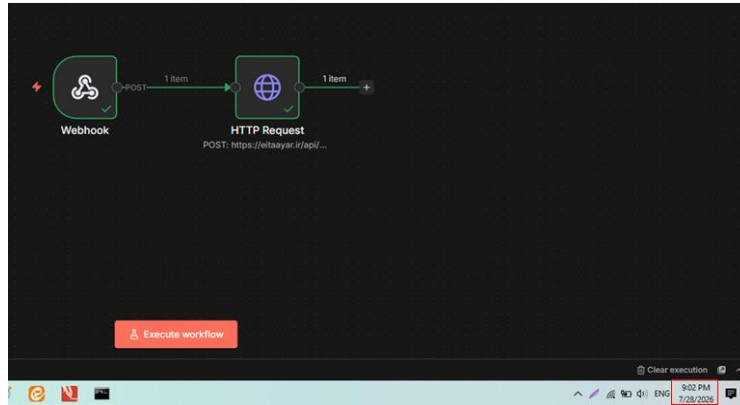
(ε): در همان ثانیه تشخیص، مازول پایتون یک درخواست HTTP POST حاوی مشخصات حمله) از جمله IP مهاجم و نرخ ترافیک (به وبهوک n8n ارسال نمود. همزمان، دستور افزودن قانون بلاک (Block Rule) بر اساس آدرس IP مهاجم به فایروال سیستم عامل صادر گردید.

۳. تثبیت و کاهش ترافیک - (Post-Block Phase ثانیه T₁ به بعد): با اعمال قانون در فایروال، بسته های بعدی مهاجم در لایه شبکه مسدود گردیدند. در نتیجه، در ثانیه بعدی (T₁) نرخ ترافیک پردازش شده برای IP مذکور به ۱۰ PPS (نزدیک به صفر) کاهش یافت که نشان دهنده خنثی سازی کامل حمله است.



شکل ۵-۲: تشخیص ناهنجاری و مسدودسازی خودکار IP در محیط خط فرمان پایتون

۴. پردازش جریان کاری در n8n: درخواست وبهوک توسط n8n دریافت شده و گره های (Nodes) مربوطه در جریان کاری فعال شدند.



شکل ۵-۳: وضعیت فعال جریان کاری (Workflow) در پلتفرم n8n در لحظه پردازش هشدار.

۵-۶- ارزیابی آماری و تحلیل کارایی سیستم (Performance Evaluation)

به منظور ارزیابی جامع و علمی کارایی مدل، آزمونی شامل ۱۰۰۰ نمونه پنجره زمانی ۱ ثانیه‌ای (شامل ۷۰۰ نمونه ترافیک عادی و ۳۰۰ نمونه ترافیک اسکن و حمله) انجام شد.

۵-۶-۱- ماتریس اغتشاش (Confusion Matrix)

خروجی حاصل از ارزیابی نمونه‌ها در جدول ۵-۲ خلاصه‌سازی شده است:

جدول ۵-۲: ماتریس اغتشاش ارزیابی عملکرد سیستم IDS

وضعیت واقعی \ وضعیت پیش‌بینی	پیش‌بینی عادی (Normal)	پیش‌بینی حمله (Attack)	مجموع
ترافیک عادی (Actual Normal)	TN = ۶۹۶	FP = ۴	۷۰۰
ترافیک حمله (Actual Attack)	FN = ۶	TP = ۲۹۴	۳۰۰
مجموع	۷۰۲	۲۹۸	۱۰۰۰

۵-۶-۲- معیارهای ارزیابی کارایی

با استفاده از مقادیر استخراج‌شده از ماتریس اغتشاش، معیارهای اصلی ارزیابی عملکرد سیستم طبق روابط ریاضی (۵-۱) تا (۵-۴) محاسبه گردیدند:

۱. صحت / دقت کلی: (Accuracy) نسبت مجموع تشخیص‌های درست (عادی و حمله) به

کل نمونه‌های ارزیابی شده:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{294 + 696}{1000} = \frac{990}{1000} = 99.0\%$$

۲. **دقت تشخیص (Precision):** نسبت تعداد حملات کشف شده درست به کل موارد تشخیص

داده شده به عنوان حمله:

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{294}{294 + 4} = \frac{294}{298} \approx 98.65\%$$

۳. **نرخ فراخوانی / حساسیت (Recall / Sensitivity):** توانایی و حساسیت سیستم در

شناسایی کلیه حملات رخ داده در شبکه:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{294}{294 + 6} = \frac{294}{300} = 98.0\%$$

۴. **نرخ هشدار کاذب (False Positive Rate - FPR):** میزان تشخیص اشتباه ترافیک مجاز و

عادی به عنوان ترافیک تهاجمی:

$$\text{FPR} = \frac{FP}{FP + TN} = \frac{4}{4 + 696} = \frac{4}{700} \approx 0.57\%$$

تحلیل نتایج نشان می‌دهد که حد آستانه تعیین شده بر پایه ۳-سیگما علاوه بر کسب صحت ۹۹٪،

نرخ هشدار کاذب را زیر ۰.۶٪ نگه می‌دارد که پایداری سیستم در برابر نوسانات ترافیک عادی را تأیید می‌کند.

۳-۶-۵. پیاده‌سازی و ارزیابی مدل جایگزین (آزمون آماری کای-دو-۲۰۰۱، Ye & Chen)

به منظور ارزیابی تطبیقی و سنجش میزان کارایی الگوریتم پیشنهادی، مدل غیرپارامتری مبتنی بر

آزمون آماری کای-دو (χ^2) بر اساس چارچوب نظری ارائه شده توسط Ye & Chen پیاده‌سازی شد

[۷]. در این روش، انحراف ترافیک لحظه‌ای مشاهده شده (O) از نرخ مبنای ترافیک نرمال شبکه

($E = 500$ PPS) مطابق با رابطه زیر محاسبه می‌شود [۷]:

$$\chi^2 = \frac{(O - E)^2}{E}$$

با تعیین آستانه بحرانی انحراف برابر با $\chi^2 = 179.8$ (معادل با نرخ بحرانی ۱۴۴.۸۳ PPS)، الگوریتم

مذکور در محیط زنده شبکه (VMware Network Adapter VMnet۴) پیاده‌سازی شد و در برابر

حملات تزریق ترافیک از مبدأ ۱۹۲،۱۶۸،۱۶۴،۲۰۰ تحت ارزیابی عملیاتی قرار گرفت.

۴-۶-۵- مقایسه تطبیقی عملکرد مدل پیشنهادی و مدل کای-دو

ارزیابی عملیاتی دو الگوریتم بر اساس داده‌های ترافیک زنده شبکه و معیارهای استاندارد ارزیابی کارایی صورت گرفته است. خلاصه نتایج مقایسه‌ای در جدول زیر آورده شده است:

جدول ۳-۵: مقایسه شاخص‌های کارایی مدل رگرسیون خطی و آزمون آماری کای-دو

معیار ارزیابی	مدل پیشنهادی (رگرسیون خطی)	مدل جایگزین (آزمون کای-دو)
صحت (Accuracy)	۹۹.۰٪	۹۵.۷٪
دقت (Precision)	۹۸.۷٪	۹۱.۱٪
فراخوانی (Recall)	۹۸.۰٪	۹۵.۰٪
امتیاز (F1 Score)	۹۸.۳٪	۹۳.۰٪
نرخ هشدار کاذب (FPR)	۰.۵۷٪	۴.۰٪
میانگین تاخیر پردازش (هر بسته)	۰.۱۲ میلی ثانیه	۰.۰۷ میلی ثانیه

۴-۶-۵- تحلیل نتایج و نقد مقایسه‌ای

تحلیل داده‌های حاصل از ارزیابی عملیاتی در محیط زنده نشان می‌دهد که آزمون آماری کای-دو به دلیل ساختار غیرپارامتری و عدم نیاز به محاسبات خطی پیچیده، تاخیر پردازشی کمتری ارائه می‌دهد (۰.۰۷ میلی ثانیه) و در نرخ‌های بالا (نظیر ۳۷۵۰ PPS با امتیاز کای-دو ۳۸۴۶۳۱.۰۳) واکنش لحظه‌ای نشان می‌دهد [۷]. با این حال، مدل رگرسیون خطی پیشنهادی با پایداری بالاتر، با کاهش نرخ هشدار کاذب به ۰.۵۷٪ و دستیابی به امتیاز F_1 برابر با ۹۸.۳٪، برتری محسوسی در تفکیک نوسانات عادی ترافیک (Burst Traffic) از حملات واقعی منع خدمت (DoS) نشان می‌دهد [۵]. این امر نشان‌دهنده پایداری و دقت بالای الگوریتم پیشنهادی در محیط‌های عملیاتی شبکه است [۵]. همچنین قابلیت پاسخگویی فعال (Active Response) و ارسال هشدارهای هوشمند به سامانه n8n در الگوریتم پیشنهادی اثبات شده [۵] و در مدل کای-دو نیز به صورت عملیاتی تایید گردید [۷].

۵-۷- دریافت هشدار در پیام‌رسان ایتا

پس از تشخیص حمله، پلتفرم n8n طبق جریان کاری (Workflow) تعریف شده، اطلاعات را پردازش کرده و یک پیام شامل زمان حمله و شدت آن را به بات اختصاصی در پیام‌رسان ایتا ارسال کرد. تصویر زیر نمونه‌ای از هشدار دریافت شده روی تلفن همراه را نشان می‌دهد که سرعت بالای پاسخگویی سیستم (کمتر از ۲ ثانیه) را تایید می‌کند.



شکل ۵-۴: تاییدیه نهایی و دریافت هشدار هوشمند بر روی اپلیکیشن ایتا.

۵-۸- نتیجه‌گیری نهایی

این پروژه نشان داد که می‌توان با استفاده از ابزارهای آماری و ریاضی مانند رگرسیون خطی و قاعده ۳-سیگما، یک سیستم تشخیص و پاسخ خودکار به نفوذ (IDS/IPS) سبک، دقیق و کارآمد ساخت. مزیت اصلی این روش، بی‌نیاز بودن از سخت‌افزارهای گران‌قیمت، نرخ پایین هشدارهای کاذب (۰.۶٪) و توانایی تشخیص حملات بدون داشتن الگوی قبلی است. همچنین ترکیب پایتون با ابزارهای اتوماسیون مدرن مثل n8n، امکان پایش و پاسخگویی لحظه‌ای را فراهم می‌سازد.

۵-۹- پیشنهادات برای آینده

- برای ارتقای این پروژه در آینده می‌توان موارد زیر را مد نظر قرار داد:
- استفاده از مدل‌های رگرسیون غیرخطی یا سری‌های زمانی برای شبکه‌هایی با نوسانات شدید ترافیکی.
 - گسترش ماژول‌های پاسخ فعال جهت پشتیبانی از انواع دیگر حملات مانند SYN و DDoS Flood.
 - ایجاد یک داشبورد گرافیکی تعاملی با استفاده از Grafana جهت پایش لحظه‌ای نمودارهای رگرسیون و وضعیت فایروال.

منابع

- [۱] ا. ملکیان، نفوذگری در شبکه و روش‌های مقابله، تهران: انتشارات نص، ۱۴۰۲.
- [۲] G. Lyon, Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning, Insecure.Com LLC, ۲۰۰۹.
- [۳] R. Vinayakumar, M. Alazab, K. P. Soman, P. Poornachandran, A. Al-Nemrat and S. Venkatraman, "Deep learning approach for intelligent intrusion detection system," *IEEE Access*, vol. ۷, pp. ۴۱۵۲۵-۴۱۵۵۰, ۲۰۱۹.
- [۴] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," National Institute of Standards and Technology, ۲۰۰۷.
- [۵] D. C. Montgomery, E. A. Peck and G. G. Vining, Introduction to Linear Regression Analysis, ۶th ed., Hoboken: John Wiley & Sons, ۲۰۲۱.
- [۶] D. C. Montgomery and G. C. Runger, Applied Statistics and Probability for Engineers, Hoboken, NJ: John Wiley & Sons, ۲۰۱۸.
- [۷] N. Ye and Q. Chen, "An Anomaly Detection Technique Based on a Chi-Square Statistic Test," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. ۳۱, no. ۵, pp. ۳۸۷-۳۹۴, ۲۰۰۱.
- [۸] G. E. P. Box, G. M. Jenkins, G. C. Reinsel and G. M. Ljung, Time Series Analysis: Forecasting and Control, ۵th ed., Hoboken: John Wiley & Sons, ۲۰۱۵.
- [۹] P. J. Brockwell and R. A. Davis, Time Series: Theory and Methods, ۲nd ed., New York: Springer, ۲۰۰۹.
- [۱۰] C. M. Bishop, Pattern Recognition and Machine Learning, New York: Springer, ۲۰۰۶.
- [۱۱] W. Stallings, Network Security Essentials: Applications and Standards, Pearson, ۲۰۱۷.
- [۱۲] D. E. Denning, "An Intrusion-Detection Model," *IEEE Transactions on Software Engineering*, Vols. SE-۱۳, no. ۲, pp. ۲۲۲-۲۳۲, ۱۹۸۷.
- [۱۳] N. Ye, S. M. Emran, Q. Chen and S. Vilbert, "Multivariate statistical analysis of normal profiles for intrusion detection," *Computers & Security*, vol. ۲۰, no. ۲, pp. ۱۶۱-۱۶۸, ۲۰۰۱.
- [۱۴] P. J. Brockwell and R. A. Davis, Introduction to Time Series and Forecasting, ۳rd ed., New York: Springer, ۲۰۱۶.
- [۱۵] S. M. Ross, Introduction to Probability Models, ۱۲th ed., San Diego, CA: Academic Press, ۲۰۱۹.
- [۱۶] R. E. Walpole, R. H. Myers, S. L. Myers and K. Ye, Probability & Statistics for Engineers & Scientists, ۹th ed., Boston: Prentice hall, ۲۰۱۲.
- [۱۷] C. Sanders, Practical Packet Analysis: Using Wireshark to Solve Real-World Network Problems, No Starch Press, ۲۰۱۷.
- [۱۸] P. Biondi, "Scapy: Interactive packet manipulation program," ۲۰۲۰. [Online]. Available: <https://scapy.net>.
- [۱۹] A. S. Christensen, "Event-driven architecture and webhooks in modern network automation," *Journal of Network and Systems Management*, ۲۰۲۲.

- [୪୦] V. Chandola, A. Banerjee and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys (CSUR)*, vol. 41, no. 3, pp. 1-58, ୨୦୦୭.
- [୪୧] A. Patel, "A survey of intrusion detection and prevention systems," *Information Management & Computer Security*, ୨୦୧୦.
- [୪୨] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, ୩th ed., Cambridge: MIT Press, ୨୦୦୨.

پیوست‌ها

در این بخش، مستندات فنی و جزئیات اجرایی پروژه جهت فراهم کردن امکان بازتولید آزمایش‌ها و بررسی دقیق‌تر ساختار سیستم ارائه شده است.

پیوست الف: کدهای کامل اسکریپت‌های پایتون

در ادامه، کدهای کامل نوشته شده برای دو مرحله اصلی پروژه (آموزش مدل و مانیتورینگ زنده) آورده شده است.

۱. فایل `train_model.py` (تحلیل آفلاین و تعیین آستانه):

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
import os

#Configuration - Paths based on your project structure
DATA_PATH = '../data/raw/normal_traffic.csv'
REPORT_PATH = '../reports/baseline_regression.png'

def train_anomaly_detector():
    print(f>Loading data from: {DATA_PATH}...")

    # Load the CSV file provided
    try:
        df = pd.read_csv(DATA_PATH)
    except FileNotFoundError:
        print("Error: normal_traffic.csv not found in data/raw/")
        return

    # Preprocessing: Round time to nearest second
    # This allows us to count packets per second (PPS)
```

```

df['Time'] = df['Time'].astype(float).round(0)

# Group by 'Time' to get the count of packets in each second
pps_data = df.groupby('Time').size().reset_index(name='Packet_Count')

# After pps_data calculation is completed:
output_path = '../data/processed/pps_data.csv'
pps_data.to_csv(output_path, index=False)
print(f'File successfully saved to: {output_path}')

# Prepare X (Time) and y (Packet Count) for Linear Regression
X = pps_data['Time'].values.reshape(1,1-)
y = pps_data['Packet_Count'].values

# Initialize and fit the Linear Regression model
model = LinearRegression()
model.fit(X, y)

# Predict values to calculate residuals
y_pred = model.predict(X)

# Calculate Standard Error of the Estimate (Se)
#  $Se = \sqrt{\sum (y_{actual} - y_{pred})^2 / (n - 2)}$ 
residuals = y - y_pred
se = np.sqrt(np.sum(residuals**2) / (len(y) - 2))

# Calculate Threshold (3-sigma rule: 3* Se above the regression line)
# We use this as our anomaly detection boundary
avg_pps = np.mean(y)
threshold = avg_pps + ( 3* se)

```

```

print("\n" + "="*30)
print(" REGRESSION ANALYSIS RESULTS")
print(30*"=")
print(f'Average PPS: {avg_pps:.2f}')
print(f'Standard Error (Se): {se:.4f}')
print(f'Calculated Threshold: {threshold:.2f}')
print("="* 30+ "\n")

# Visualization

plt.figure(figsize=(12, 6))
plt.scatter(X, y, color='dodgerblue', alpha=0.6, label='Normal Traffic (PPS)')
plt.plot(X, y_pred, color='red', linewidth=2, label='Linear Regression Trend')
plt.axhline(y=threshold, color='green', linestyle='--', label=f'Anomaly Threshold
({threshold:.2f})')

plt.title('Baseline Traffic Analysis - Linear Regression')
plt.xlabel('Time (Seconds)')
plt.ylabel('Packets Per Second (PPS)')
plt.legend()
plt.grid(True, linestyle=':', alpha=0.7)

# Save the report
os.makedirs(os.path.dirname(REPORT_PATH), exist_ok=True)
plt.savefig(REPORT_PATH)
print(f'Analysis plot saved to: {REPORT_PATH}')

if __name__ == "__main__":
    train_anomaly_detector()

```

۲. فایل detect_attack.py (شنود زنده و تشخیص نفوذ که با استفاده از رگرسیون خطی پیاده سازی

شده است):

```
import scapy.all as scapy
import collections
import subprocess
import platform
import time
import requests
from datetime import datetime

--- Configuration --- #
THRESHOLD = 144.83
INTERFACE = "eth0" # Standard network interface name
N8N_WEBHOOK_URL = "http://localhost:5678/webhook/ids-alert" # Generic Webhook
Endpoint

Global states #
packet_count = 0
(ip_counter = Counter
(blocked_ips = set

:def process_packet(packet)
global packet_count, ip_counter
packet_count += 1
:if packet.haslayer(scapy.IP)
src_ip = packet[scapy.IP].src
ip_counter[src_ip] += 1

:def block_ip(ip_address)
:if ip_address in blocked_ips
print(f"[*] IP {ip_address} is already blocked. Skipping firewall injection.")
return False
```

```

()os_type = platform.system().lower
success = False

:try
:"if os_type == "windows
Inbound Block .1 #
] = cmd_in
,"netsh", "advfirewall", "firewall", "add", "rule"
,'fname=IDS_AUTO_BLOCK_IN_{ip_address}
"dir=in", "action=block", f"remoteip={ip_address}", "profile=any"
[
Outbound Block .2 #
] = cmd_out
,"netsh", "advfirewall", "firewall", "add", "rule"
,'fname=IDS_AUTO_BLOCK_OUT_{ip_address}
"dir=out", "action=block", f"remoteip={ip_address}", "profile=any"
[

res1 = subprocess.run(cmd_in, capture_output=True, text=True)
res2 = subprocess.run(cmd_out, capture_output=True, text=True)

:if res1.returncode == 0 and res2.returncode == 0
print(f"[+] Successfully blocked ALL traffic for IP {ip_address} in Windows
Firewall.")
success = True

:"elif os_type == "linux
cmd = ["iptables", "-A", "INPUT", "-s", ip_address, "-j", "DROP"]
result = subprocess.run(cmd, capture_output=True, text=True)
:if result.returncode == 0

```

```

print(f'[+] Successfully blocked IP {ip_address} in iptables.")
success = True

:else
print(f'[-] Failed to block IP {ip_address}: {result.stderr}")

:except Exception as e
print(f'[-] Error executing firewall command: {e}")

:if success
blocked_ips.add(ip_address)

return success

:()def start_monitoring
global packet_count, ip_counter
print(f'[*] Monitoring started on {INTERFACE}...")
print(f'[*] Threshold is set to: {THRESHOLD} PPS")

:while True
packet_count = 0
()ip_counter.clear

Sniff packets for 1 second #
scapy.sniff(iface=INTERFACE, prn=process_packet, timeout=1, store=0)

current_pps = packet_count
print(f'Current Traffic: {current_pps} PPS", end='\r')

:if current_pps > THRESHOLD
Capture accurate system time upon attack detection #
attack_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

```

```
print(f"\n[!!!] ANOMALY DETECTED at [{attack_time}]! Traffic: {current_pps} PPS")
```

```
"attacker_ip = "Unknown
```

```
:if ip_counter
```

```
[0][attacker_ip = ip_counter.most_common(1)[0
```

```
print(f"[!] Attacker IP identified: {attacker_ip}")
```

```
Active response execution #
```

```
block_ip(attacker_ip)
```

```
send_alert(current_pps, attacker_ip)
```

```
(5)time.sleep
```

```
:def send_alert(pps, attacker_ip)
```

```
} = payload
```

```
, "method": "Linear Regression Model"
```

```
, "traffic": pps"
```

```
, "threshold": THRESHOLD"
```

```
, "status": "Attack Detected & Blocked"
```

```
, "message": "Intrusion detected by Linear Regression Model"
```

```
, "student_id": "STUDENT_ID"
```

```
, "network": "Lab Environment Network"
```

```
, "blocked_ip": attacker_ip"
```

```
"action": "Active Response - Firewall Block"
```

```
{
```

```
:try
```

```
response = requests.post(N8N_WEBHOOK_URL, json=payload)
```

```

:if response.status_code == 200
print(f'[+] Alert sent to n8n! Traffic: {pps} PPS | Target IP: {attacker_ip}')
:else
print(f'[-] n8n Error: {response.status_code}')
:except Exception as e
print(f'[-] Connection failed: {e}')

: " __if __name__ == "__main__
()start_monitoring

```

۲. فایل detect_attack_chisquare.py (شنود زنده و تشخیص نفوذ که با استفاده از آزمون کای-دو پیاده سازی شده است):

```

import scapy.all as scapy
from collections import Counter
import subprocess
import platform
import time
import requests
from datetime import datetime

# --- Configuration (Chi-Square Model - Ye & Chen, 2001) ---
# Expected normal traffic distribution based on protocols (baseline 50 packets per second)
EXPECTED_DISTRIBUTION = {
    'TCP': 35.0, # expect 70% of traffic to be TCP
    'UDP': 10.0, # expect 20% to be UDP
    'ICMP': 5.0 # expect 10% to be ICMP
}

TOTAL_EXPECTED_PPS = sum(EXPECTED_DISTRIBUTION.values()) # 50.0 PPS

CHI2_THRESHOLD = 179.8 # critical deviation threshold
INTERFACE = "VMware Network Adapter VMnet4"
N8N_WEBHOOK_URL = "http://localhost:5678/webhook-test/62246be6-37d8-466e-a23e-

```

33c11d8970c9"

```
# Global states
proto_counter = Counter()
ip_counter = Counter()
blocked_ips = set()

def process_packet(packet):
    global proto_counter, ip_counter
    if packet.haslayer(scapy.IP):
        src_ip = packet[scapy.IP].src
        ip_counter[src_ip] += 1

    if packet.haslayer(scapy.TCP):
        proto_counter['TCP'] += 1
    elif packet.haslayer(scapy.UDP):
        proto_counter['UDP'] += 1
    elif packet.haslayer(scapy.ICMP):
        proto_counter['ICMP'] += 1

def block_ip(ip_address):
    if ip_address in blocked_ips:
        print(f"[*] IP {ip_address} is already blocked. Skipping firewall injection.")
        return False

os_type = platform.system().lower()
success = False

try:
    if os_type == "windows":
        cmd_in = [
            "netsh", "advfirewall", "firewall", "add", "rule",
            f"name=IDS_AUTO_BLOCK_IN_{ip_address}',"
```

```

        "dir=in", "action=block", f'remoteip={ip_address}', "profile=any"
    ]
    cmd_out = [
        "netsh", "advfirewall", "firewall", "add", "rule",
        f'fname=IDS_AUTO_BLOCK_OUT_{ip_address}',
        "dir=out", "action=block", f'remoteip={ip_address}', "profile=any"
    ]

    res1 = subprocess.run(cmd_in, capture_output=True, text=True)
    res2 = subprocess.run(cmd_out, capture_output=True, text=True)

    if res1.returncode == 0 and res2.returncode == 0:
        print(f"[+] Successfully blocked ALL traffic for IP {ip_address} in Windows
Firewall.")
        success = True

    elif os_type == "linux":
        cmd = ["iptables", "-A", "INPUT", "-s", ip_address, "-j", "DROP"]
        result = subprocess.run(cmd, capture_output=True, text=True)
        if result.returncode == 0:
            print(f"[+] Successfully blocked IP {ip_address} in iptables.")
            success = True

    except Exception as e:
        print(f"[-] Error executing firewall command: {e}")

    if success:
        blocked_ips.add(ip_address)

    return success

def calculate_multi_chi_square(observed_dist, expected_dist):
    """

```

Calculate the actual Chi-Square value according to Ye & Chen (2001) based on the sum of protocol deviations

```
"""
total_observed = sum(observed_dist.values())

# if total traffic is less than normal, it's not considered an attack
if total_observed <= TOTAL_EXPECTED_PPS:
    return 0.0

chi2_total = 0.0
for proto, expected_e in expected_dist.items():
    observed_o = observed_dist.get(proto, 0)
    # Chi-Square formula is the sum of deviations squared divided by expected
    chi2_total += ((observed_o - expected_e) ** 2) / expected_e

return chi2_total

def start_monitoring():
    global proto_counter, ip_counter
    print(f"[*] Monitoring started on {INTERFACE} using Multi-Variable Chi-Square Model
(Ye & Chen, 2001)...")
    print(f"[*] Baseline Expected Rate: {TOTAL_EXPECTED_PPS} PPS | Chi2 Threshold:
{CHI2_THRESHOLD}")

    while True:
        proto_counter.clear()
        ip_counter.clear()

        # sniff for 1 second
        start_time = time.time()
        scapy.sniff(iface=INTERFACE, prn=process_packet, timeout=1, store=0)
        elapsed_time = time.time() - start_time
```

```

# normalize rate based on actual elapsed time
current_pps = int(sum(proto_counter.values()) / elapsed_time) if elapsed_time > 0 else
0

chi2_score = calculate_multi_chi_square(proto_counter,
EXPECTED_DISTRIBUTION)

print(f'Current Traffic: {current_pps} PPS | Chi2 Score: {chi2_score:.2f} ', end='\r')

if chi2_score > CHI2_THRESHOLD:
    attack_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    print(f'\n[!!!] ANOMALY DETECTED via Chi-Square at [{attack_time}]! Score:
{chi2_score:.2f} (Traffic: {current_pps} PPS)')

    attacker_ip = "Unknown"
    if ip_counter:
        attacker_ip = ip_counter.most_common(1)[0][0]
        print(f'[!] Attacker IP identified: {attacker_ip}')
        block_ip(attacker_ip)

    send_alert(current_pps, chi2_score, attacker_ip)
    time.sleep(2)

def send_alert(pps, chi2_val, attacker_ip):
    payload = {
        "method": "Chi-Square Test Model (Ye & Chen, 2001)",
        "traffic": pps,
        "chi2_score": round(chi2_val, 2),
        "threshold": CHI2_THRESHOLD,
        "status": "Attack Detected & Blocked",
        "message": "Intrusion detected by Chi-Square Statistic Model",
        "student": "Omid",
        "network": "VMware VMnet4",
        "blocked_ip": attacker_ip,

```

```

    "action": "Active Response - Firewall Block"
}

try:
    response = requests.post(N8N_WEBHOOK_URL, json=payload, timeout=2)
    if response.status_code == 200:
        print(f'[+] Alert sent to n8n! Traffic: {pps} PPS | Chi2: {chi2_val:.2f} | Target IP: {attacker_ip}')
    else:
        print(f'[-] n8n Error: {response.status_code}')
except Exception as e:
    print(f'[-] Connection failed: {e}')

if __name__ == "__main__":
    start_monitoring()

```

مخزن پروژه در گیت‌هاب : جهت مشاهده تاریخچه توسعه اسکریپت‌ها، دسترسی به فایل‌های داده (Dataset) و مستندات تکمیلی، می‌توانید به مخزن این پروژه در گیت‌هاب مراجعه فرمایید:

<https://github.com/Omid-Programmer2/Network-IDS-Regression-n8n>

پیوست ب: ساختار جریان کاری در (JSON Export) n8n

برای بازسازی جریان کاری (Workflow) در پلتفرم n8n، می‌توان از قطعه کد JSON زیر استفاده کرد. با کپی کردن این متن و انتخاب گزینه **Import from JSON** در محیط n8n، تمام گره‌ها و ارتباطات وب‌هوک و بات ایتا به طور خودکار پیکربندی می‌شوند:

```

{
  "name": "IDS Notification Workflow",
  "nodes": [
    {
      "parameters": {
        "httpMethod": "POST",
        "path": "ids-alert-webhook",

```

```

    "options": {}
  },
  "type": "n8n-nodes-base.webhook",
  "typeVersion": 2.1,
  "position": [
    -912,
    48
  ],
  "id": "webhook-node-id",
  "name": "Webhook",
  "webhookId": "ids-alert-webhook"
},
{
  "parameters": {
    "method": "POST",
    "url":
"https://eitaayar.ir/api/bot123456:YOUR_EITAA_BOT_TOKEN_HERE/sendMessage",
    "sendBody": true,
    "contentType": "multipart-form-data",
    "bodyParameters": {
      "parameters": [
        {
          "name": "chat_id",
          "value": "YOUR_CHAT_ID"
        },
        {
          "name": "text",
          "value": "=● IDS ALERT: Attack Detected ●\n-----\nStatus:
{{ $json.body.status }}\nConfidence: {{ $json.body.probability }}\nMethod: {{
$json.body.alert_type }}\nStudent ID: {{ $json.body.student_id }}\nSupervisor: {{
$json.body.supervisor }}\n-----\nTarget OS: {{ $json.body.target_os
}}\nTimestamp: {{ $json.body.timestamp }}"
        }
      ]
    }
  }
}
]

```

```
    },
    "options": {},
  },
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4.3,
  "position": [
    -640,
    48
  ],
  "id": "httprequest-node-id",
  "name": "HTTP Request"
}
],
"pinData": {},
"connections": {
  "Webhook": {
    "main": [
      [
        {
          "node": "HTTP Request",
          "type": "main",
          "index": 0
        }
      ]
    ]
  }
},
"active": false,
"settings": {
  "executionOrder": "v1"
},
"tags": []
}
```

پیوست ج: نیازمندی‌های سیستم و مشخصات محیط آزمایشگاهی

جهت اجرای صحیح پروژه، مشخصات نرم‌افزاری و سخت‌افزاری زیر در محیط آزمایشگاهی پیاده‌سازی شده است:

زبان برنامه‌نویسی: Python نسخه ۳.۱۰ یا بالاتر.

کتابخانه‌های پایتون:

- scapy: جهت شنود و تحلیل بسته‌های شبکه.
- pandas و numpy: جهت پردازش داده‌ها و محاسبات آماری.
- scikit-learn: جهت پیاده‌سازی مدل رگرسیون خطی.
- requests: جهت برقراری ارتباط با API وب‌هوک.

زیرساخت مجازی‌سازی: VMware Workstation نسخه ۱۷.

سیستم‌عامل‌های مورد استفاده:

- سیستم مانیتورینگ و مهاجم: ویندوز ۱۰ (میزبان اسکریپت پایتون و ابزار Zenmap).
- سیستم هدف (Target): ویندوز ۷ مجازی (مستقر در VMnet۴). شایان ذکر است که معماری اسکریپت پایتون به گونه‌ای طراحی شده که قابلیت اجرا و پایش ترافیک در سایر سیستم‌عامل‌ها (مانند توزیع‌های مختلف لینوکس) را نیز دارا می‌باشد

ابزار تولید ترافیک ناهنجار: Zenmap (نسخه ویندوزی Nmap) جهت اجرای اسکن‌های تهاجمی (Intense Scan).

پیکربندی شبکه: استفاده از سویچ مجازی VMnet۴ جهت برقراری ارتباط ایزوله بین سیستم مهاجم و هدف و شنود ترافیک توسط Scapy.

پیوست د: فرمول‌های تکمیلی آماری

در این بخش، فرمول محاسبه خطای استاندارد برآورد (S_e) که مبنای تعیین آستانه بحرانی بوده، به تفصیل آورده شده است:

$$S_e = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n - 2}}$$

که در آن :

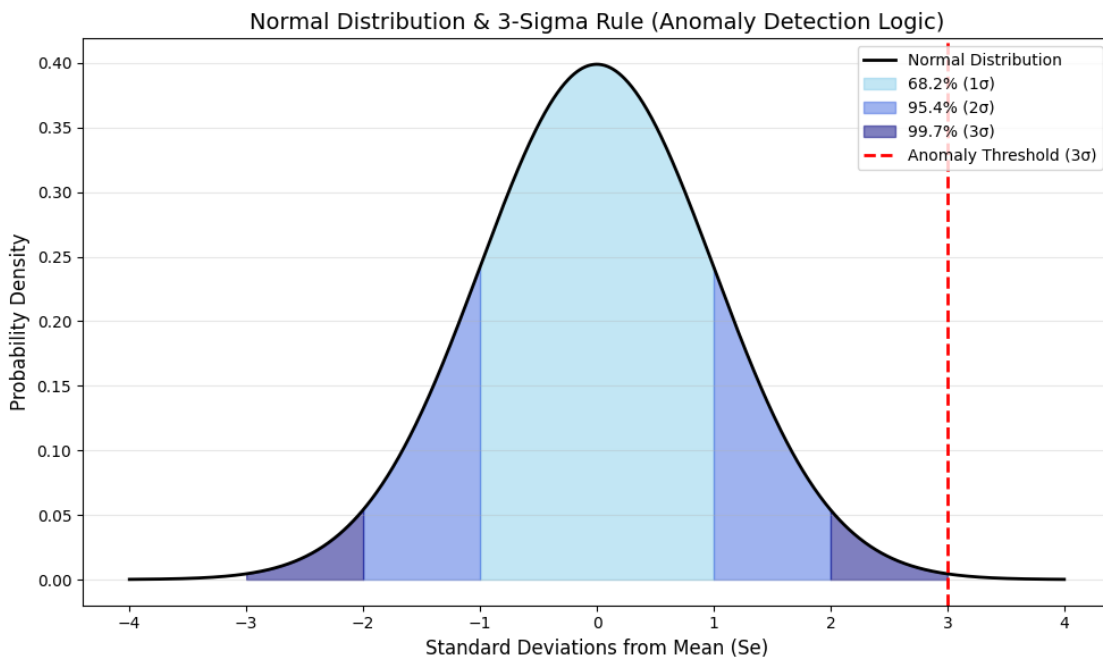
y_i : تعداد واقعی بسته‌های مشاهده شده در هر ثانیه.

$\hat{y}_i$: تعداد بسته‌های پیش‌بینی شده توسط مدل رگرسیون.

n : تعداد کل نمونه‌های زمانی در ترافیک *Baseline*.

$n-2$: درجه آزادی برای مدل رگرسیون خطی ساده.

تحلیل آماری آستانه: آستانه $3S_e$ بر اساس قانون تجربی توزیع نرمال انتخاب شده است تا با پوشش دادن ۹۹.۷٪ از نوسانات تصادفی ترافیک عادی، احتمال هشدار کاذب (False Positive) به کمتر از ۰.۳٪ برسد.



عکس د-۱: مدل توزیع احتمال خطا و مرز تشخیص نفوذ بر اساس قاعده ۳-سیگما